

2024-2025

SAÉ 3

RECETTE 1

LUCCI & DALIGAUX

```
Microsoft Windows [version 10.0.22631.4662]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\geii>ping 192.168.1.60

Envoi d'une requête 'Ping' : 192.168.1.60 avec 32 octets de données :
Réponse de 192.168.1.60 : octets=32 temps=7 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=6 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=70 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=15 ms TTL=255

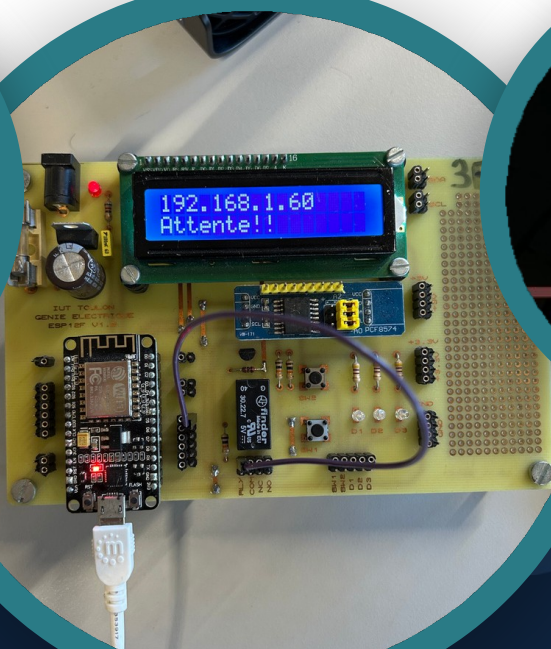
Statistiques Ping pour 192.168.1.60 :
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 6ms, Maximum = 70ms, Moyenne = 30ms

C:\Users\geii>ping lumen.local

Envoi d'une requête 'ping' sur lumen.local [192.168.1.60] avec 32
Réponse de 192.168.1.60 : octets=32 temps=7 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=1 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=0 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=16 ms TTL=255

Statistiques Ping pour 192.168.1.60 :
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte
    Durée approximative des boucles en millisecondes :
        Minimum = 7ms, Maximum = 16ms, Moyenne = 6ms

C:\Users\geii>
```



est Unitaire PWM

Commande Relai

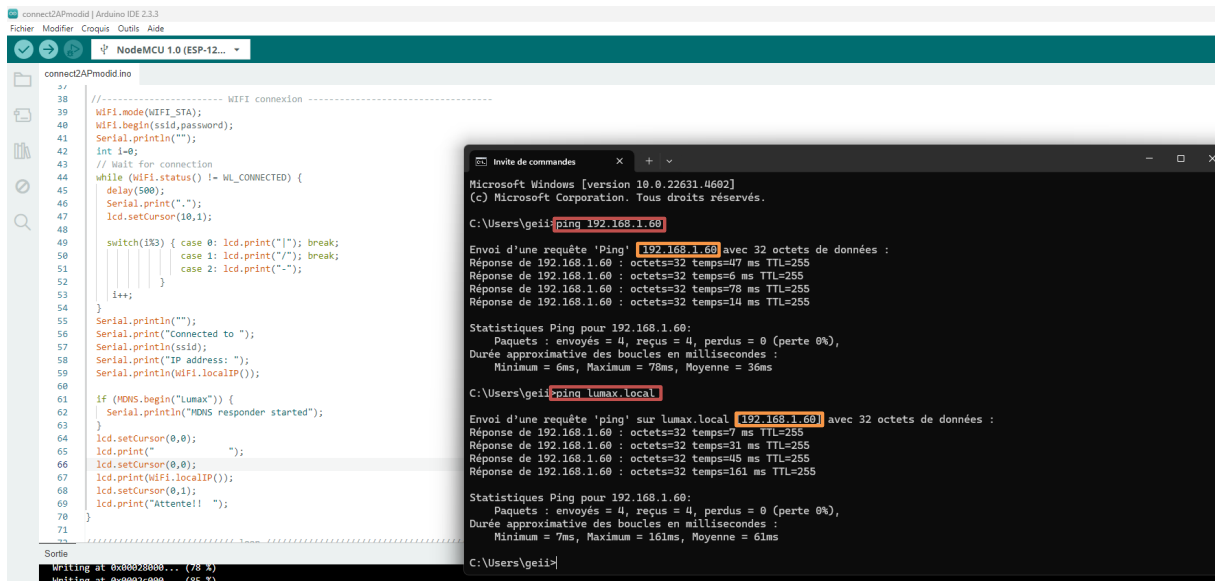
Mettre à 0 Allumer au max

Statut du Relai : OFF

PWM

Valeur du PWM : 0% Actualiser PWM

TU1 : Connection Access Point.



L'objectif de ce test est de trouver l'adresse IP de notre serveur web et de vérifier la connectivité ordinateur serveur web. Les critères de réussite de ce test Unitaire sont : l'affichage de l'adresse IP sur l'écran LCD, vérification de l'adresse IP récupéré et ainsi que test de connectivité.



Le test est effectué avec un ping sur l'IP affichée sur L'écran CD afin de vérifier la connectivité de cette adresse et ensuite un nouveau ping sur l'url du serveur web "Lumax.local", ce ping vérifie que l'url du serveur correspond bien à l'IP affichée.

Les commandes suivantes vont permettre d'établir le ssid ainsi que le mot de passe de la connexion à laquelle il faut se connecter. Donc ici e307 en ssid et comme mot de passe geiigeii.

```
#ifndef STASSID
#define STASSID "e307"
#define STAPSK "geiigeii"
#endif

const char* ssid = STASSID;
const char* password = STAPSK;
```

```
//----- WIFI connexion -----
WiFi.mode(WIFI_STA);
WiFi.begin(ssid,password);
Serial.println("");
int i=0;
// Wait for connection
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
  lcd.setCursor(10,1);

  switch(i%3) { case 0: lcd.print("|"); break;
               case 1: lcd.print("/"); break;
               case 2: lcd.print("-"); break;
               }

  i++;
}
Serial.println("");
Serial.print("Connected to ");
Serial.println(ssid);
Serial.print("IP address: ");
Serial.println(WiFi.localIP());
```

La connexion wifi est établie par les commandes Wifi.mode ainsi que wifi.begin en entrant le ssid et le mot de passe. L'url du serveur web est définie dans MDNS.begin celle choisie sera donc Lumax.local. Par la suite, l'adresse IP est récupéré avec Wifi.localIP() et affiché sur l'écran lcd avec un lcd.print(Wifi.localIP()). Donc l'adresse IP est bien affiché sur l'écran LCD désormais il faut vérifier l'IP et sa connectivité pour valider le test unitaire.

```

if (MDNS.begin("Lumax")) {
  Serial.println("MDNS responder started");
}

lcd.setCursor(0,0);
lcd.print("          ");
lcd.setCursor(0,0);
lcd.print(WiFi.localIP());
lcd.setCursor(0,1);
lcd.print("Attente!! ");

```

TU2 : Génération PWM 30KHz

L'objectif du test unitaire 2 va être de générer un signal PWM variant d'une fréquence de 0 à 30 KHz qu'on affiche sur le Pin pour ainsi le visualiser à l'oscilloscope.

```

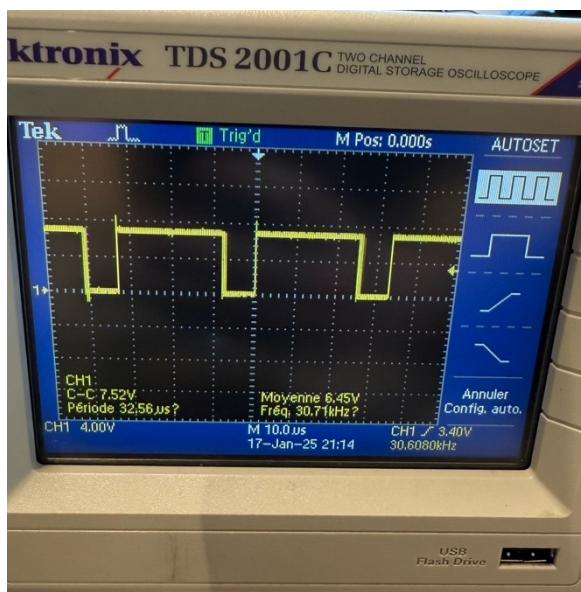
const int pwmFrequency = 30000;
int pwmval = 0, pwmMax = 255;

//////////////////// setup //////////////////////
void setup(void) {
  analogWriteFreq(pwmFrequency);
  pinMode(PwmPin, OUTPUT);
}

//////////////////// loop //////////////////////
void loop(void) {
  for(;pwmval < pwmMax ; pwmval++){analogWrite(PwmPin,pwmval);delay(15);}
  for(;pwmval > 0 ; pwmval--){analogWrite(PwmPin,pwmval);delay(15);}
}

```

Les commandes vont permettre de créer un signal PWM de 30000 Hz. La valeur de PWM est variée de 0 à 255 qui par la suite est affiché sur le Pin du PWM permettant ainsi de le visualisé sur un oscilloscope.



Grâce à la visualisation sur l'oscilloscope du signal présent sur le Pin du PWM, il est possible d'observer la variation de la valeur du PWM et ainsi vérifier son bon fonctionnement.

Le test unitaire fonctionne correctement, et le PWM varie bien lorsqu'il est visualisé sur l'oscilloscope.

TU3 : Page WEB avec des boutons et/ou un slider

L'objectif du test unitaire 3 va être de créer la page WEB en HTML, de pouvoir gérer le signal PWM à partir d'un slider et/ou boutons et d'afficher le retour de la valeur du PWM sur l'écran LCD.

La fonction `getPage()` renvoie la chaîne de caractères avec le String qui contient le contenu HTML de la page Web. La page commence avec les balises de base d'un document HTML

```
"<head>";
String getPage() {
    String page = "<html>";
    page += "<head>";
    page += "<title>Test Unitaire WEB</title>";
    page += "</head>";
    page += "<body>";
    // Header
    page += "<header>";
    page += "<h1>Test Unitaire WEB</h1>";
    page += "</header>";
    // Main content container
    page += "<div class='container'>";

    page += "<form action='/' method='POST'>";
    page += "<button type='submit' name='RLY' value='0'>Mettre a 0 </button>";
    page += "<button type='submit' name='RLY' value='1'>Allumer au max</button>";
    page += "</form>";
    // Slider container
    page += "<div class='slider-container'>";
    page += "<h3> WEB </h3>";
    page += "<form action='/' method='POST'>";
    page += "<input type='range' name='Variable' min='0' max='100' value='\" +
String(pwmval) + \"' oninput='this.nextElementSibling.value = this.value'>";
    page += "<output>\" + String(pwmval) + \"</output>";
    page += "<div class='slider-label'>Valeur du PWM : <strong>\" + String(pwmval)
+ \"</strong></div>";
    page += "<button type='submit'>Actualiser variable</button>";
    page += "</form>";
    page += "</div>";
    page += "</body>";
    page += "</html>";
    return page;
}
```

```
page += "<title>Test Unitaire WEB</title>";
page += "</head>";
page += "<body>";
```

Cela définit ainsi le titre de la page web qui est ici Test Unitaire WEB et prépare le corp du document.

L'entête est rajouté à la suite avec un titre principal.

```
page += "<header>";
page += "<h1>Test Unitaire WEB</h1>";
page += "</header>";
```

Une requête post est envoyé à l'URL racine ('/') avec les boutons qui transmettent la valeur (value) pour la variable RLY. Le 0 pour éteindre le relais et mettre le PWM à la valeur 0 sur le slider qui sera expliquer par la suite, le 1 qui permet d'allumer le relais et ainsi mettre la valeur du PWM au maximum.

```
page += "<div class='container'>";
```

```
page += "<form action='/' method='POST'>";
page += "<button type='submit' name='RLY' value='0'>Mettre a 0 </button>";
page += "<button type='submit' name='RLY' value='1'>Allumer au max</button>";
page += "</form>";
```

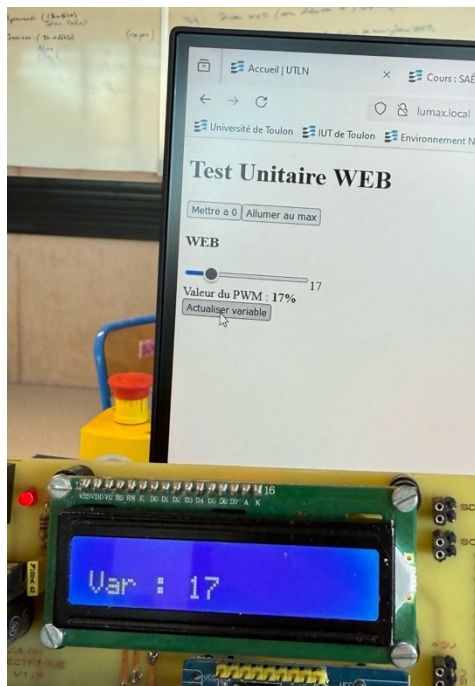
Le slider va permettre de sélectionner une valeur entre 0 et 100 avec la fonction `slider(input type='range')`. La valeur actuelle est définie par la variable `pwmval` qui est convertie en chaîne avec un string par la suite. Lorsqu'on modifie le slider, la valeur s'affiche dynamiquement dans un champ associé grâce à l'attribut `oninput`.

```
page += "<div class='slider-container'>";
page += "<h3> WEB </h3>";
page += "<form action='/' method='POST'>";
page += "<input type='range' name='Variable' min='0' max='100' value='" + String(pwmval) +
"" oninput='this.nextElementSibling.value = this.value'>";
page += "<output>" + String(pwmval) + "</output>";
page += "<div class='slider-label'>Valeur du PWM : <strong>" + String(pwmval) +
"%</strong></div>";
page += "<button type='submit'>Actualiser variable</button>";
```

Un bouton 'actualiser variable' permet d'injecter la valeur du slider dans le PWM, permettant ainsi de régler la valeur du PWM avant de l'injecter.

Les dernières fonctions permettent de finir la mise en page et de mettre fin à la structure HTML.

```
page += "</form>";
page += "</div>";
page += "</div>";
page += "</body>";
page += "</html>";
```



Ici la valeur du PWM sur le slider est de 17%, après avoir injecter la valeur de celui-ci, l'ecran LCD affiche la valeur du PWM qui est bien égal à celui sur la page Web. Le test unitaire est validé car le retour de la valeur du PWM se fait correctement.

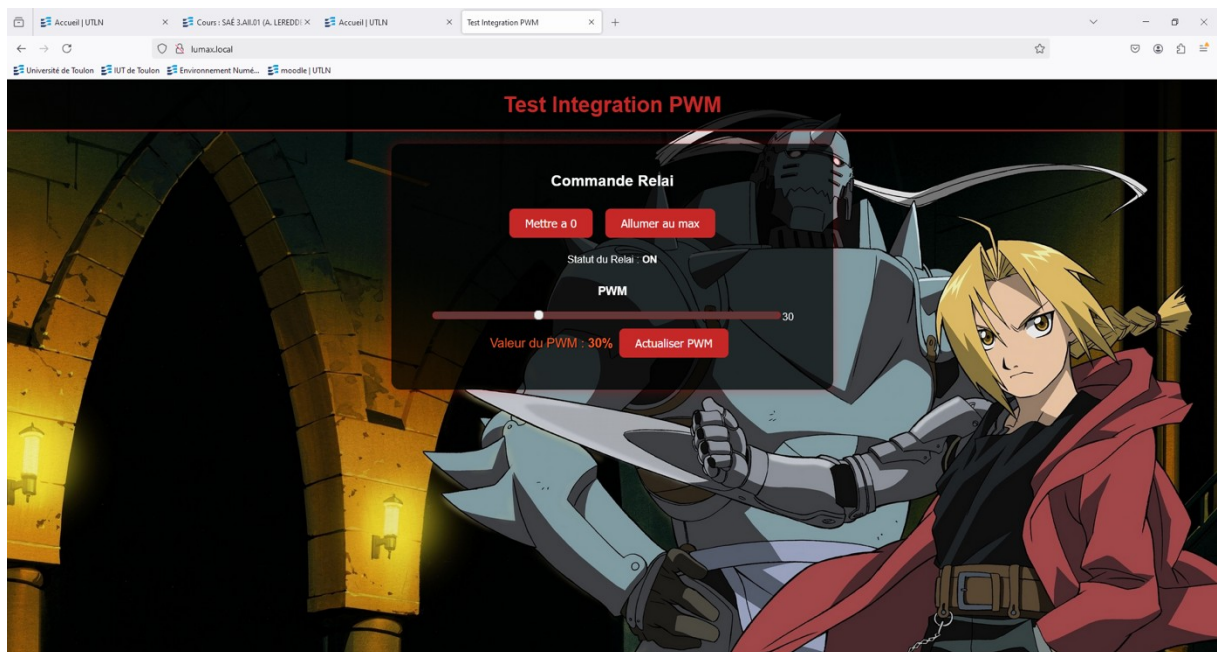
TI1 : Page WEB et génération PWM

L'objectif du test intégration va être de créer la page WEB en HTML, de pouvoir gérer le signal PWM à partir d'un slider et/ou boutons ainsi que le relais et d'afficher le retour de la valeur du PWM sur l'écran LCD. Pour finir le visualiser sur l'oscilloscope.


```
String getPage() {
    String page = "<html>";
    page += "<head>";
    page += "<title>Test Integration PWM</title>";
    page += "<style>";
    // Body styling
    page += "body { background-image: url('https://wallpaperswide.com/download/fullmetal_chemist_manga-wallpaper-2560x1440.jpg');"; // Replace with your
    // desired Fullmetal Alchemist image link
    page += "background-size: cover; background-attachment: fixed; color:
    #ffffff; font-family: 'Arial', sans-serif; margin: 0; padding: 0; }";
    page += "header { background-color: rgba(0, 0, 0, 0.85); color: #c62828;
    text-align: center; padding: 20px; font-size: 28px; border-bottom: 2px solid
    #c62828; }";
    page += "h1 { font-size: 34px; margin: 0; text-shadow: 2px 2px 5px #000000; }
    ";
    // Container styling
    page += ".container { background-color: rgba(0, 0, 0, 0.75); margin: 20px
    auto; padding: 25px; border-radius: 15px; max-width: 650px; text-align:
    center; box-shadow: 0 0 15px rgba(255, 0, 0, 0.3); }";
    page += "button { background-color: #c62828; color: #ffffff; border: none;
    padding: 12px 25px; font-size: 18px; cursor: pointer; border-radius: 8px;
    margin: 10px; transition: all 0.2s ease-in-out; }";
    page += "button:hover { transform: scale(1.1); background-color: #e53935; }";
    // Slider styling
    page += ".slider-container { margin-top: 30px; }";
    page += "input[type='range'] { -webkit-appearance: none; width: 85%;
    background: #444; height: 12px; border-radius: 6px; outline: none;
    box-shadow: inset 0 0 10px rgba(255, 0, 0, 0.6); }";
    page += "input[type='range']:ebkit-slider-thumb { -webkit-appearance: none;
    width: 30px; height: 30px; background: radial-gradient(circle, #fff000 60%,
    #c62828); border-radius: 50%; cursor: pointer; box-shadow: 0 0 15px rgba(255,
    0, 0, 0.8); }";
    page += ".slider-label { font-size: 20px; color: #ff5722; margin-top: 10px;
    display: inline-block; text-shadow: 1px 1px 2px #000; }";
    // Footer styling
    page += ".footer { color: #c62828; font-size: 14px; margin-top: 30px;
    text-shadow: 1px 1px 2px #000000; }";
    page += "</style>";
    page += "</head>";
    page += "<body>";
    // Header
    page += "<header>";
    page += "<h1>Test Integration PWM</h1>";
    page += "</header>";
```

Le principe de la WEB est le même que pour le TU3 dans le fonctionnement des boutons ainsi que le slider. Le programme a été modifié pour rajouter de l'esthétique à la page WEB en rajoutant des couleurs, centrant les titres, rajoutant un fond d'écran à partir d'un lien d'image.

```
// Main content container
page += "<div class='container'>";
page += "<h2>Commande Relai</h2>";
page += "<form action='/' method='POST'>";
page += "<button type='submit' name='RLY' value='0'>Mettre a 0 </button>";
page += "<button type='submit' name='RLY' value='1'>Allumer au max</button>";
page += "</form>";
page += "<div class='status'>Statut du Relai : <strong>" + String
(stateRelay ? "ON" : "OFF") + "</strong></div>";
// Slider container
page += "<div class='slider-container'>";
page += "<h3> PWM </h3>";
page += "<form action='/' method='POST'>";
page += "<input type='range' name='PWM' min='0' max='100' value='" + String
(pwmval) + "' oninput='this.nextElementSibling.value = this.value'>";
page += "<output>" + String(pwmval) + "</output>";
page += "<div class='slider-label'>Valeur du PWM : <strong>" + String(pwmval)
+ "%</strong></div>";
page += "<button type='submit'>Actualiser PWM</button>";
page += "</form>";
page += "</div>";
page += "</div>";
page += "</body>";
page += "</html>";
return page;
}
```



La page s'affiche ainsi, avec les éléments centrés, des couleurs appliquées, et un fond d'écran personnalisé sélectionné.

Les fonctions suivantes sont modifiées afin d'ajouter l'état du relais avec la variable stateRelay qui passe à HIGH à partir du moment où le PWM dépasse 1% et repasse à LOW quand la valeur du PWM est 0. Il rentre dans if selon l'état du relais LOW/HIGH est écrit sur

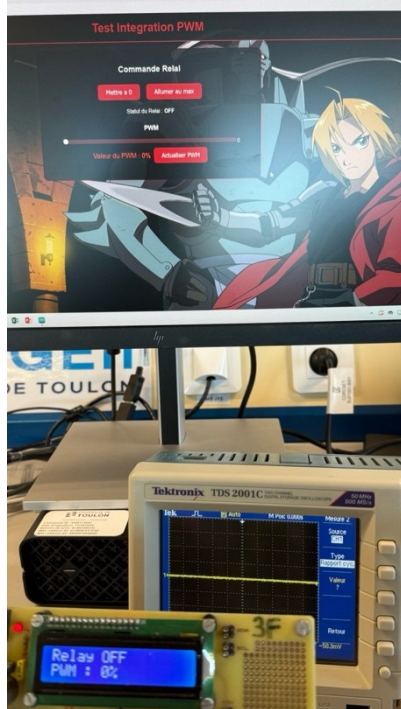
l'écran LCD Relay OFF/Relay ON.

```
void handleRoot(){
  String postValue;
  if ( server.hasArg("RLY")) {
    postValue = server.arg("RLY");
    if ( postValue == "1" ) {
      stateRelay = HIGH;
      pwmval = 100;
      pwmcalc = 255;
      digitalWrite(cmdeRelay, stateRelay);
      lcd.setCursor(0,0);
      lcd.print("Relay ON ");
      server.send ( 200, "text/html", getPage() );
    } else
      if ( postValue == "0" ) {
        stateRelay = LOW;
        pwmval = pwmcalc= 0;
        digitalWrite(cmdeRelay, stateRelay);
        lcd.setCursor(0,0);
        lcd.print("Relay OFF");
        server.send ( 200, "text/html", getPage() );
      }
  }
  if (server.hasArg("PWM")) {
    pwmval = server.arg("PWM").toInt();
    if(pwmval >= 1){stateRelay = HIGH;digitalWrite(cmdeRelay,
stateRelay);lcd.setCursor(0,0);lcd.print("Relay ON ");}
    analogWriteFreq(pwmFrequency);
    pwmcalc = pwmval*2.55;
    analogWrite(PwmPin,pwmcalc);
    lcd.setCursor(0,1);
    lcd.print("PWM : ");
    lcd.setCursor(8,1);
    lcd.print("  ");
    lcd.setCursor(6,1);
    lcd.print(pwmval);
    lcd.print("%");
    server.send ( 200, "text/html", getPage() );
  }

  server.send ( 200, "text/html", getPage() );
}
```



Lors du premier test, la valeur du PWM est réglée à 4 %, ce qui met à jour l'écran LCD et la page WEB en affichant correctement l'état du relais, passé en ON, ainsi que la valeur exacte du PWM en %. Sur la broche PWM, l'oscilloscope détecte bien un signal correspondant à 4 %. Le deuxième fonctionne aussi correctement avec l'état du relais qui est bien en ON ainsi que la valeur de 36% du PWM qui s'affiche bien sur l'oscilloscope.



Lors du dernier test, la valeur du PWM injectée est de 0 %. L'état du relais est passé en OFF, ce qui est correctement affiché sur l'écran LCD ainsi que sur la page web. La valeur 0 % est bien indiquée sur l'écran LCD. En conséquence, l'oscilloscope ne détecte aucun signal sur la broche PWM.

En conclusion, les trois tests ont été validés avec succès. Les valeurs du PWM et l'état du relais s'affichent correctement sur l'écran LCD et la page web pour chaque scénario. L'oscilloscope confirme la cohérence des signaux PWM injectés, avec une détection fidèle des signaux pour les valeurs supérieures à 0 % et une absence de signal pour un PWM de 0 %. Ces résultats attestent du bon fonctionnement du système dans toutes les configurations testées.

Le test intégration 1, qui intègre la gestion du relais, du signal PWM, et l'affichage via une page web et un écran LCD, répond pleinement à son objectif. Les différents tests réalisés ont confirmé le bon fonctionnement des fonctionnalités : les valeurs du PWM sont correctement manipulées via les boutons ou le slider, l'état du relais est affiché avec précision, et les signaux

PWM sont visualisés avec exactitude sur l'oscilloscope. La mise en page web, avec ses éléments esthétiques et fonctionnels, offre une expérience utilisateur claire et intuitive. L'ensemble valide que l'intégration est réussie.